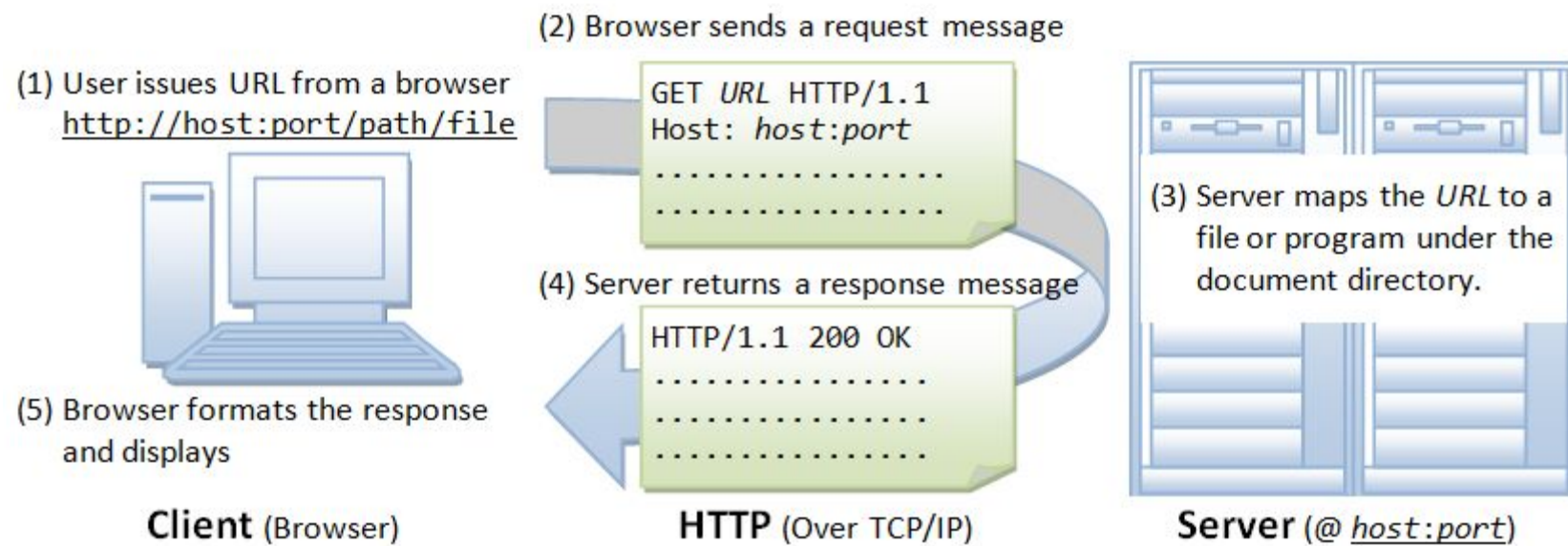


# HTTP

dr. Matej Šprogar

# Odjemalec / strežnik arhitektura



# HyperText Transfer Protocol

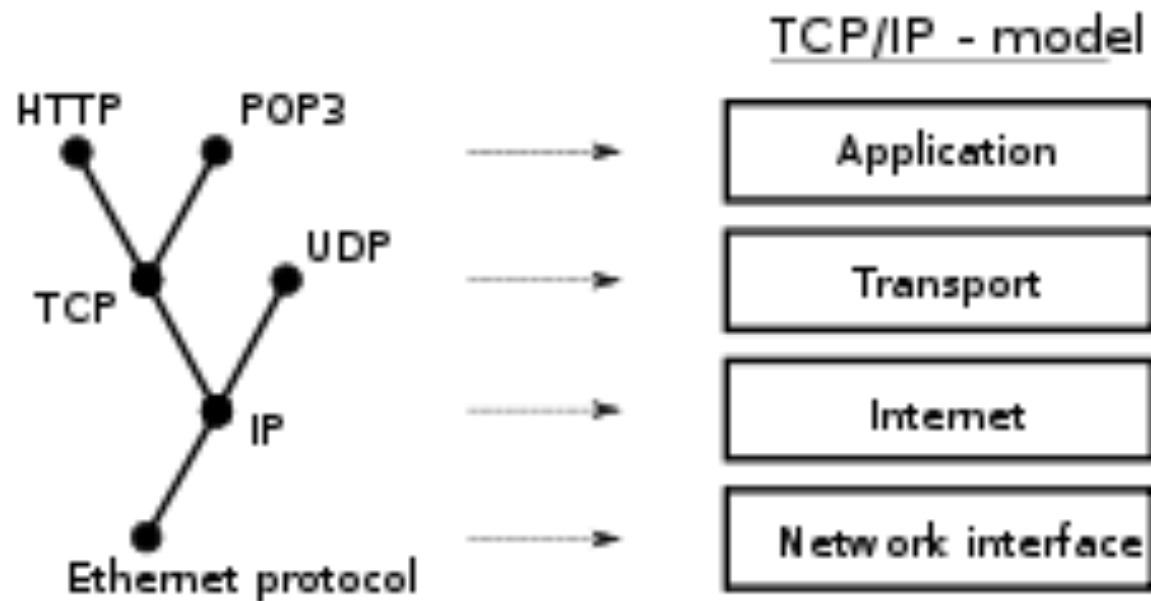
<https://developer.mozilla.org/en-US/docs/Web/HTTP>

- Aplikacijski protokol za prenos (hipertekstovnih) dokumentov (HTML, XML, XHTML, pa tudi txt, jpg...)
- HTTP je protokol tipa zahteva/odgovor
- Protokol brez stanja
  - zahteve na isti povezavi *niso* avtomatsko povezane
  - FTP pa je recimo protokol s stanjem (strežnik hrani nastavitve v okviru posebne *kontrolne* povezave)
  - podatkov o seji strežnik ne vodi avtomatsko v okviru HTTP protokola, ampak po potrebi na višjem nivoju

Verzije:

- HTTP/0.9 je bil enostaven, HTTP/1.0 pozna MIME, headerje  
HTTP/1.1 hitrejši, trajne povezave, HTTP/2 multiplexing, HTTP/3  
QUIC+UDP namesto TCP

# Komunikacijski protokoli



# URL *Uniform Resource Locator*

URL določa, kako lahko dostopamo do vira in kje se vir nahaja

- Sintaksa: protokol + strežnik [+ vrata] + pot [+parametri + fragment]
- Tudi "spletni naslov" (*web address*)
- "strežnik(domena)" + "vrata" == "authority"
- Vsi URLji morajo biti URL *enkodirani*
- URL primeri
  - <http://www.mojadomena.si/index.html>
  - <https://www.mojadomena.si:8080/index.html#logo>
  - <ftp://ftp.mojadomena.si/primer.txt>
- Protokoli
  - http, https, ftp, file, news, mailto, telnet, ...

# URL Encoding

URLji, ki jih pošiljamo v okviru HTTP zahtev, ne morejo neposredno vsebovati rezerviranih (npr. ' ', '=', '&', ...) in kontrolnih (prvih 32) znakov.

Vsaj te znake moramo zato pretvoriti v univerzalno šestnajstiško kodo znaka v znakovni tabeli:

Primeri:

|                 |   |                      |
|-----------------|---|----------------------|
| ' ' (presledek) | → | %20 (lahko tudi '+') |
| '+'             | → | %2B                  |
| '='             | → | %3D                  |
| '&'             | → | %26                  |
| '%'             | → | %25                  |

glej [https://www.w3schools.com/tags/ref\\_urlencode.ASP](https://www.w3schools.com/tags/ref_urlencode.ASP)

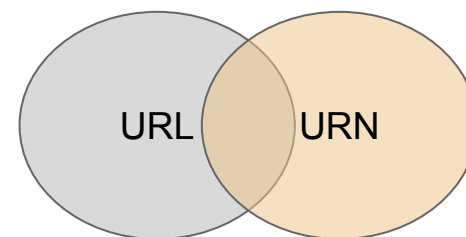
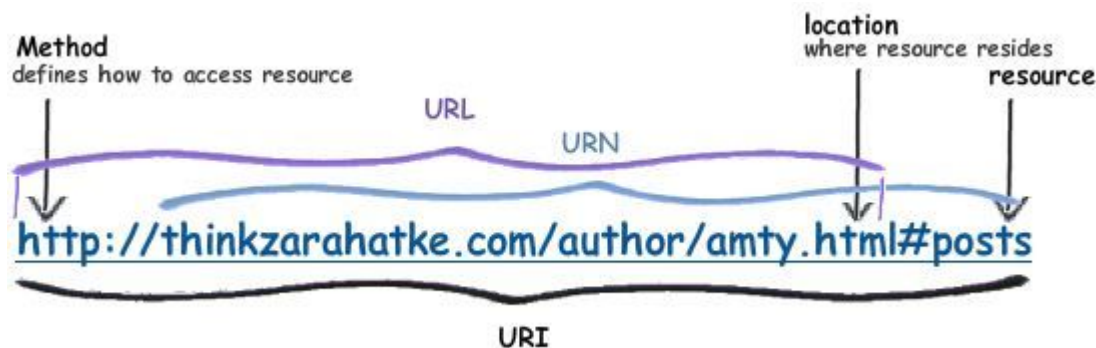
# URI = URL + URN?

- URIdentifier - identificira vir ali po imenu ali lokaciji ali oboje
- URLocator - identificira vir z lokacijo; vsak URL je tudi URI
- URName - identificira vir z unikatnim imenom; je tudi URI

URI: `http://www.nekaj.com/vir` (content negotiation!)

URL: `http://www.nekaj.com/vir.jpg`

URN: `urn:isbn:978-1-491-92706-9`



URI = URL + URN

# HTTP zahteva

- Odjemalec (tipično browser) se poveže na HTTP strežnik, privzeto preko porta 80
- Odjemalec pošlje zahtevo (HTTP ukaz)

METODA          Request-URI          HTTP-Verzija  
[zaglavje (*headerji...*)]  
(prazna vrstica)  
[morebitna vsebina]

```
GET /docs/index.html HTTP/1.1
Host: www.somewhere.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)
(blank line)
```



# HTTP metode/ukazi

HTTP/0.9

***GET***

HTTP 1.0

***POST, HEAD***

HTTP 1.1

***PUT, DELETE, OPTIONS, TRACE, CONNECT***

*razširitve: PATCH*

*druge: PROPFIND, COPY, MOVE, SEARCH, REPORT*

# Idempotentnost HTTP ukazov

*Isti ukaz je lahko narejen večkrat, vendar **ponovitev** ukaza ne povzroči **novih** posledic glede na **prvo** izvršitev.*

- Idempotentni ukaz lahko ima server-side posledice!
- *GET, PUT, DELETE* so po naravi idempotentni
- Naštete ukaze pa lahko implementiramo tudi v ne-idempotentni obliki!
- Pomembno za varnost v poglavju o piškotih (*SameSite LAX*)

# Zakaj idempotentnost

## 1. Zanesljivost pri ponovitvah (retry logic)

V primeru, ko uporabnik klikne gumb "Shrani" in povezava za trenutek pade, je pomembna narava uporabljene metode.

✚ Če se uporablja metoda PUT (ki je idempotentna), je zahtevo mogoče varno ponoviti brez tveganja za ustvarjanje podvojenih zapisov.

✗ Če se uporabi POST, lahko ponovitev povzroči nastanek dvojnika.

## 2. Caching (predpomnjenje)

Brskalniki, proxy strežniki in CDN-ji lahko predpomnijo idempotentne metode (npr. GET in HEAD), saj pri njih ni nevarnosti, da bi se zaradi zahteve spremenilo stanje na strežniku.

## 3. Varnost in logika API-jev

Metode v API-jih naj bodo oblikovane dosledno in predvidljivo. Če dokumentacija predvideva uporabo metode PUT na naslovu /user/123, mora vsak tak klic povzročiti enako stanje vira, ne glede na to, kolikokrat se zahteva ponovi.

## 4. REST arhitektura

REST arhitekturni slog temelji na predpostavki, da so nekatere metode idempotentne, druge pa ne. Upoštevanje teh načel prispeva k bolj čisti zasnovi in zmanjšuje število robnih primerov oziroma težav.

# HTTP GET

- Namenjen za pridobivanje podatkov s strežnika
- Tipično zahteva "reprezentacijo" izbranega vira (*resource*), ponavadi je to kar dokument ali dinamični odziv strežnika na zahtevo (po tem viru).
- GET *naj bi* zgolj pridobil podatke, brez stranskih učinkov
- Dejansko z GET tudi prenesemo podatke na strežnik
  - podatek je konec koncev že to, kateri dokument zahtevamo
  - v URL zahteve pa lahko vključimo še več podatkov:
    - začnejo se z znakom ?
    - zakodirani so kot pari `ime=vrednost`
    - ločeni so z znakom &
  - Podatki morajo biti URL enkodirani\*
- Ukaz lahko služi kot *zaznamek*.
- Primer:

`http://www.google.com/search?q=Java+XML&start=10`

# HTTP POST

- Kreiranje vira na strežniku
- Namenjen za posredovanje podatkov izbranemu viru na strežniku
- Podatki so vključeni v telesu zahteve
- Prenašamo lahko večje količine podatkov
- V telesu zahteve je lahko tekstovna ali binarna vsebina

Primer:

```
POST /test/demo.php HTTP/1.1
```

```
Host: w3schools.com
```

```
name1=value1&name2=value2
```

# Drugi ukazi

- **PUT**  
Spremeni vir na strežniku - zamenja obstoječo vsebino izbranega URL-ja z vsebino telesa sporočila
- **DELETE**  
Izbriše z URL-jem določen dokument, vir
- **HEAD**  
isto kot GET, vendar strežnik vrne zgolj glavo, brez vsebine
- **OPTIONS**  
Vprašanje strežniku, kakšne HTTP metode razume
- **TRACE**  
Vrne zahtevo po poti skozi požarne zidove in proxyje
- **CONNECT**  
Ustvari HTTP tunel
- **PATCH** (ni vedno podprt)  
Namenjen manjšemu popravku vira na strežniku

# POST ali PUT?

| Lastnost          | POST            | PUT                     |
|-------------------|-----------------|-------------------------|
| Namen             | Ustvari nov vir | Ustvari ali zamenja vir |
| Lokacija (URL)    | Določi strežnik | Določi odjemalec        |
| Idempotentnost    | Ne              | Da                      |
| Varnost ponovitev | Lahko podvoji   | Vedno enak rezultat     |
| Uporaba za update | Manj pogosto    | Pogosto                 |

POST: Dodajanje novega zapisa brez vnaprej določenega identifikatorja,  
Pošiljanje podatkov za obdelavo brez pričakovane lokacije

PUT: Dodajanje novega zapisa z vnaprej določenim identifikatorjem  
Posodabljanje obstoječega zapisa

# HTTP/1.1 Odgovor strežnika

Odgovor vsebuje:

1. Protokol + StatusnaKoda + OpisStatusa
2. Zaglavje (množica spremnih informacij)
3. - prazno vrstico -
4. Telo (dejanska vsebina)

```
HTTP/1.1 200 OK
```

```
Date: Sun, 18 Oct 2009 08:56:53 GMT
```

```
Server: Apache/2.2.14 (Win32)
```

```
Last-Modified: Sat, 20 Nov 2004 07:16:26 GMT
```

```
ETag: "10000000565a5-2c-3e94b66c2e680"
```

```
Accept-Ranges: bytes
```

```
Content-Length: 44
```

```
Connection: close
```

```
Content-Type: text/html
```

```
X-Pad: avoid browser bug
```

```
<html><body><h1>It works!</h1></body></html>
```



# HTTP/2/3 Odgovor strežnika

Ker sta HTTP/2 in HTTP/3 binarna protokola, se header ne pošilja več v tekstovni obliki, ampak v posebnem, kompresiranem HEADERS okvirju (*frame*).

Dodatno so uvedeni posebni headerji (*:method*, *:path*, *:scheme*, *:authority*), ki nadomeščajo starejši format; prednost je manjša količina podatkov in s tem kompresija ter lažje parsanje, saj ni problemov s presledki in vrsticami in drugimi napakami pri branju.

Če je bil HTTP/1.1 header zgolj seznam vrstic, je binarni header strukturiran binarni objekt, za katerega razumevanje nujno potrebujemo parser.

# Kategorije HTTP/1.1 statusnih kod

## **1xx**    *Informational*

Zahteva sprejeta, proces se nadaljuje.

## **2xx**    *Uspeh*

Zahteva je bila uspešno sprejeta in izvedena.

## **3xx**    *Redirekcija*

Za dokončanje zahteve so potrebni dodatni koraki

## **4xx**    *Uporabniška napaka*

Zahteva je slabo oblikovana ali ne more biti izpolnjena

## **5xx**    *Strežniška napaka*

Strežnik ni izvedel sicer veljavne zahteve

# Pomembnejše kode:

|     |                       |
|-----|-----------------------|
| 200 | OK                    |
| 201 | Created               |
| 202 | Accepted              |
| 300 | Multiple Choices      |
| 301 | Moved Permanently     |
| 304 | Not Modified          |
| 400 | Bad Request           |
| 401 | Unauthorized          |
| 404 | Not Found             |
| 409 | Conflict              |
| 414 | URI Too Long          |
| 500 | Internal Server Error |
| 503 | Service Unavailable   |

# HTTPS = HTTP Secure

HTTP ni varen, ker podatki potujejo kot navadno besedilo in jih lahko prestreže posrednik (MITM napad).

HTTPS = HTTP + TLS (šifriranje), uporablja port **443**, ščiti podatke med prenosom.

## Kako HTTPS deluje?

1. klient vzpostavi povezavo
2. strežnik pošlje **certifikat**
3. klient preveri identiteto
4. dogovor o ključih (TLS handshake)
5. začne se **šifrirana komunikacija**

## Kaj zagotavlja TLS (HTTPS)?

- ✓ enkripcijo (nihče drug ne vidi vsebine komunikacije med računalnikoma)
- ✓ integriteto (vsebine prispejo na cilj nespreminjene)
- ✓ avtentikacijo (brskalnik dokazano komunicira s pravim strežnikom glede na ime)